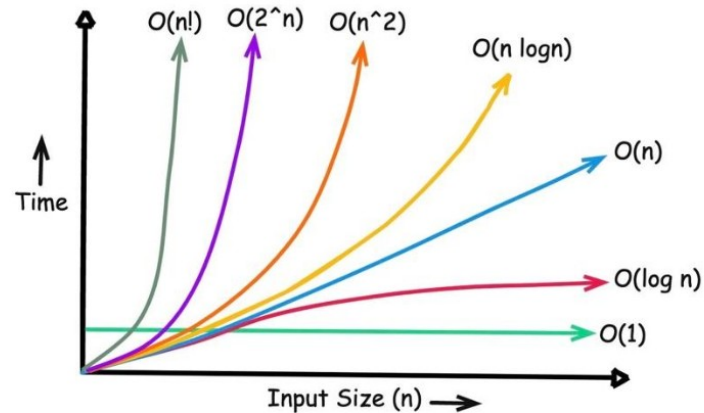


# Lecture 8

## Theoretical Running Time III



# Announcements

- ❖ **Office hours** are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group
- ❖ **Project 2** Planning & Analysis due **Friday @ 11:59pm**
  - See [example project](#) on Github
  - Be sure to comment your Github link on the project document

# Today

- ❖ Other asymptotic bounds
  - Big- $\Omega$
  - Big- $\Theta$

# Common Bounds

- ❖ Constant time:  $O(1)$
- ❖ Log time:  $O(\log n)$
- ❖ Linear time:  $O(n)$
- ❖ Log Linear time:  $O(n \log n)$
- ❖ Quadratic time:  $O(n^2)$
- ❖ Cubic time:  $O(n^3)$
- ❖ Polynomial time:  $O(n^k)$  for some  $k > 0$
- ❖ Exponential time:  $O(2^n)$

Big-O is an  
upper bound

Faster Running Time  
Slower Growth



Slower Running Time  
Faster Growth

# Common Bounds

- ❖ Constant time:  $O(1)$
- ❖ Log time:  $O(\log n)$
- ❖ Linear time:  $O(n)$
- ❖ Log Linear time:  $O(n \log n)$
- ❖ Quadratic time:  $O(n^2)$
- ❖ Cubic time:  $O(n^3)$
- ❖ Polynomial time:  $O(n^k)$  for some  $k > 0$
- ❖ Exponential time:  $O(2^n)$

Big-O is an  
upper bound

Faster Running Time  
Slower Growth

Slower Running Time  
Faster Growth

# New Ideas

## ❖ Big- $\Omega$

- Lower bound on the worst case running time of an algorithm
- Why is this useful?

# New Ideas

## ❖ Big- $\Omega$

- Lower bound on the worst case running time of an algorithm
- Why is this useful?

## ❖ Big- $\Theta$

- An exact characterization of asymptotic running time

# Big-Ω Definition

Let  $T(n)$  be the number of steps required for our algorithm in the worst case

**Definition:** We say that  $T(n)$  is  $\Omega(f(n))$  if there exists a  $c > 0$  such that

$$T(n) \geq cf(n)$$

for sufficiently large  $n$

- ❖ Then  $f$  is an **asymptotic lower bound** of  $T$
- ❖ **Informally:**  $T$  grows as least as fast as  $f$

# Big-Ω Definition

Let  $T(n)$  be the number of steps required for our algorithm in the worst case

**Definition:** We say that  $T(n)$  is  $\Omega(f(n))$  if there exists a  $c > 0$  such that

$$T(n) \geq cf(n)$$

for sufficiently large  $n$

- ❖ Then  $f$  is an **asymptotic lower bound** of  $T$
- ❖ **Informally:**  $T$  grows as least as fast as  $f$
- ❖ How does this definition differ from Big-O?

# Big-Ω Definition

Let  $T(n)$  be the number of steps required for our algorithm in the worst case

**Definition:** We say that  $T(n)$  is  $\Omega(f(n))$  if there exists a  $c > 0$  such that

$$T(n) \geq cf(n)$$

for sufficiently large  $n$

- ❖ Then  $f$  is an **asymptotic lower bound** of  $T$
- ❖ **Informally:**  $T$  grows as least as fast as  $f$
- ❖ How does this definition differ from Big-O?
  - The direction of the inequality

# Examples

- ❖ If  $T(n) = 4n + 10$ , then  $T(n)$  is  $\Omega(n)$

# Examples

- ❖ If  $T(n) = 4n + 10$ , then  $T(n)$  is  $\Omega(n)$
- ❖ If  $T(n) = 0.5 * n^3$ , then  $T(n)$  is  $\Omega(n^3)$

# Examples

- ❖ If  $T(n) = 4n + 10$ , then  $T(n)$  is  $\Omega(n)$
- ❖ If  $T(n) = 0.5 * n^3$ , then  $T(n)$  is  $\Omega(n^3)$
- ❖ If the best-case an algorithm takes  $2n$  steps, then it is  $\Omega(n)$

# Common Bounds

- ❖ Constant time:  $\Omega(1)$
- ❖ Log time:  $\Omega(\log n)$
- ❖ Linear time:  $\Omega(n)$
- ❖ Log Linear time:  $\Omega(n \log n)$
- ❖ Quadratic time:  $\Omega(n^2)$
- ❖ Cubic time:  $\Omega(n^3)$
- ❖ Polynomial time:  $\Omega(n^k)$  for some  $k > 0$
- ❖ Exponential time:  $\Omega(2^n)$

Big- $\Omega$  is a  
lower bound

Faster Running Time  
Slower Growth



Slower Running Time  
Faster Growth

# Common Bounds

- ❖ Constant time:  $\Omega(1)$
- ❖ Log time:  $\Omega(\log n)$
- ❖ Linear time:  $\Omega(n)$
- ❖ Log Linear time:  $\Omega(n \log n)$
- ❖ Quadratic time:  $\Omega(n^2)$
- ❖ Cubic time:  $\Omega(n^3)$
- ❖ Polynomial time:  $\Omega(n^k)$  for some  $k > 0$
- ❖ Exponential time:  $\Omega(2^n)$

Big- $\Omega$  is a  
lower bound

Faster Running Time  
Slower Growth



Slower Running Time  
Faster Growth

# Big- $\Theta$ Definition

Let  $T(n)$  be the number of steps required for our algorithm in the worst case

**Definition:** We say that  $T(n)$  is  $\Theta(f(n))$  if  $T(n)$  is  $O(f(n))$  and  $T(n)$  is  $\Omega(f(n))$

- ❖ Then  $f$  is an **asymptotic tight bound** for  $T$
- ❖ **Informally:**  $T$  grows just as fast as  $f$

# Exercise 1

Let  $T(n) = 3n^2 + 17n + 2$ . Which of the following are true?

a)  $T(n)$  is  $\Theta(n^2)$

b)  $T(n)$  is  $\Theta(n)$

c)  $T(n)$  is  $\Theta(n^3)$

# Exercise 1

Let  $T(n) = 3n^2 + 17n + 2$ . Which of the following are true?

a)  $T(n)$  is  $\Theta(n^2)$

b)  $T(n)$  is  $\Theta(n)$

c)  $T(n)$  is  $\Theta(n^3)$

# How to use these bounds

- ❖ Analyze your algorithm
- ❖ If you can calculate  $T(n)$ , use it to prove a Big- $\Theta$  bound
- ❖ Some algorithms are very difficult to analyze!
- ❖ You may be able to upper/lower bound  $T(n)$ 
  - Use the upper bound to find a potentially loose Big- $O$  bound
  - Use the lower bound to find a potentially loose Big- $\Omega$  bound

# Common Running Times

- ❖ Constant time:  $\Theta(1)$
- ❖ Log time:  $\Theta(\log n)$
- ❖ Linear time:  $\Theta(n)$
- ❖ Log Linear time:  $\Theta(n \log n)$
- ❖ Quadratic time:  $\Theta(n^2)$
- ❖ Cubic time:  $\Theta(n^3)$
- ❖ Polynomial time:  $\Theta(n^k)$  for some  $k > 0$
- ❖ Exponential time:  $\Theta(2^n)$

Faster Running Time  
Slower Growth



Slower Running Time  
Faster Growth

# Next Time

- ❖ Asymptotic Lower & Exact Bounds
- ❖ DSP 5.4-5.10